

Memoria PEC 1 C.E.

Algoritmo Generativo Sudoku

por Jaime Ciriaco Beaumont

Introducción

Los algoritmos evolutivos representan un grupo de métodos de optimización basados en la evolución biológica expresada por Charles Darwin, tales como la selección natural, recombinación genética, mutación, mejores individuos, etc. Entre estos algoritmos, de las variantes más conocidas tenemos al algoritmo genético (AG), el cual trabaja con poblaciones de soluciones y por su carácter estocástico. Es por esto que trabajaremos con él para desarrollar la práctica explicada a continuación.

El objetivo de esta PEC es aplicar el tipo de algoritmo AG para tratar de resolver un Sudoku, un ejercicio de comprensión y análisis, pues además de tratar de resolver el juego de forma aislada se pide que analicemos rendimientos: eficacia y eficiencia del modelo desarrollado. Todo este apartado de análisis al igual que la producción del código están basados con el respaldo de los conocimientos brindados en los capítulos 1,2,4,9 y 14 del libro “Fundamentos de la Computación Evolutiva” por Enrique J.Carmona Suárez y Severino Fernández Galán.

Específicamente como pide el enunciado, nos centraremos en investigar la influencia de la probabilidad de mutación dentro del modelo, iterando el desarrollo de sudokus con diferentes valores de mutación.

Descripción del Problema del Sudoku.

El Juego matemático Sudoku, no es más que una matriz, que almacena huecos y números enteros entre el 1 y el 9, normalmente sus medidas son de 9x9. Esto definiría el tablero inicial, pero el juego reside en que mediante unas reglas sencillas se puede lograr rellenar por completo la matriz. Estas reglas son:

- No se puede repetir ningún número en una fila
- No se puede repetir ningún número en una columna
- No se puede repetir ningún número en una sub-cuadrícula 3x3

En el enunciado se consideran 2 instancias del problema:

- Sudoku fácil, con mayor número de celdas inicialmente rellenas
- Sudoku difícil, con menor información inicial y mayor dificultad

A partir de estos analizaremos su comportamiento con respecto a diferentes variantes de mutación y además veremos si hay disimilitud entre sudoku fácil y difícil debido a la diferida complejidad.

Implementación

El algoritmo genérico que implementamos sigue el esquema canónico descrito en el capítulo 1 del libro, pues trabaja con su flujo y etapas explicadas a continuación:

Representación de individuos

Cada individuo (cadena de $9 \times 9 = 81$ valores) representa una posible solución candidata al Sudoku, representando al genotipo del AG, donde el gen sería cada celda y sus posibles valores serían enteros entre 1 y 9, además las celdas iniciales del Sudoku se consideran celdas fijas. Las celdas “vacías” son representadas por el valor 0 y se diferencian así para tenerlas en cuenta y más adelante seleccionar el resto para no modificarlas durante la evolución del algoritmo.

Función de adaptación (fitness)

La función que usamos en la implementación es la dada por el enunciado donde: $f(i)$ es el número de conflictos en la misma fila, $c(i)$ es el número de conflictos en la misma columna y $s(i)$ es el número de conflictos en la misma sub-cuadrícula 3x3 teniendo así cubiertas todas las reglas del sudoku base.

La fórmula completa la cual definiría el fenotipo se vería así:

$$\frac{\sum_{i=1}^{81} \{f(i) + c(i) + s(i)\}}{2}$$

Siendo esta la naturaleza de la fórmula, el mejor caso corresponde a $\text{fitness} = 0$, el cual indica que el sudoku se soluciona en su totalidad sin errores, por esto cuanto mayor sea el fitness indicará un mayor número de conflictos.

Iniciación de la población

Para la población inicial teníamos la libertad de generar de forma aleatoria los valores del gen en el genotipo pero, pudiendo representar las restricciones iniciales del sudoku desde la primera población, lo cual ayudaría a reducir mucho el primer fitness, nos decidimos por aplicar esta lógica.

El flujo de trabajo en este apartado sería; hacer una primera instancia de que celdas fijas para conservar en el futuro su valor inicial y para cada celda vacía seleccionar un valor permitido aplicando el fitness.

Dentro de los problemas planteados en los enunciados para el Sudoku fácil se usan 10 individuos y para el Sudoku difícil se usan 100 individuos (al igual que con el Sudoku base el cual se plantea y resuelve en el código donde se está haciendo toda esta lógica)

Selección de padres

Tal y como dicta el enunciado, usamos selección por torneo, en la que se seleccionan de forma aleatoria varios individuos de la población y seleccionamos como padre el que haya dado el mejor(menor) valor de fitness.

De esta forma permitimos controlar la presión selectiva y además, después de diferentes pruebas se estimó que el valor más acertado sería un torneo de 4 individuos, compitiendo muy a la par con un torneo de 3 individuos, pues aunque de 3 a 4 haya una diferencia notoria en tiempos, aumenta respectivamente también la probabilidad de sacar un mejor fitness.

Operadores de variación

Cruce: Aplicamos cruce por un punto en el genotipo de los padres con una probabilidad de cruce del 90%. Para asegurar, tras el cruce, se aplican las celdas fijas otra vez sobre la matriz para evitar que por un error se pierda su valor original.

Mutación: La mutación se hace por gen o celda, con una probabilidad de mutación base del 3%. Cuando la mutación se produce solo se modifican las celdas no fijas y se vuelve a dar un valor permitido por fitness. Si la mutación no se da, se igualan directamente padres e hijos.

Selección de supervivientes

Utilizamos un modelo generacional donde toda la población es reemplazada por los hijos. Como añadido, aplicamos elitismo, asegurándonos de que el mejor individuo de la generación anterior no se pierda si las futuras generaciones, después de los operadores de variación, empeoran.

Metodología de evaluación

Como ya mencionamos en la introducción, el AG tiene un carácter estocástico, por eso una única ejecución del programa como se hace en el código “ga_sudoku_base.py” no es representativo para tratar de analizar el modelo. Por ahora tenemos este código el cual es recipiente de toda la lógica base que hemos planteado con anterioridad, la lógica del esquema canónico, pero, nos queda hacer un sistema donde, aprovechando esta base, iteramos la resolución de Sudokus para analizar más tarde su eficacia.

Aquí es donde entra “ga_sudoku_evaluation.py” el cual comprende los Sudokus que pide el enunciado (sudoku_facil y sudoku_dificil), opcionalmente a partir de un menú por consola los resuelve de forma aislada en una única ejecución y también comprende la opción de iterar y generar un csv con los resultados de aplicar a la resolución del sudoku facil o dificil diferentes valores de mutación. En este caso las opciones que plantea la iteración en valores de mutación son:

0.005, 0.01, 0.02, 0.03, 0.05, 0.1, 0.2, 0.5

El número de ejecuciones trabajado para cada valor de mutación es de 20, lo cual en el campo de la ciencia no es considerado ni una muestra pequeña ni grande, más bien ajustada para el modelo que estamos trabajando.

En la evaluación del modelo buscamos métricas que puedan evaluar tanto la calidad de las soluciones como la velocidad de convergencia que tiene el algoritmo, por ello, por cada iteración de mutación sacamos: la tasa de éxito (TE), el valor medio relativo sumando todos los individuos finales dividido por el número de generaciones (VAMM) y las generaciones medias hasta el éxito.

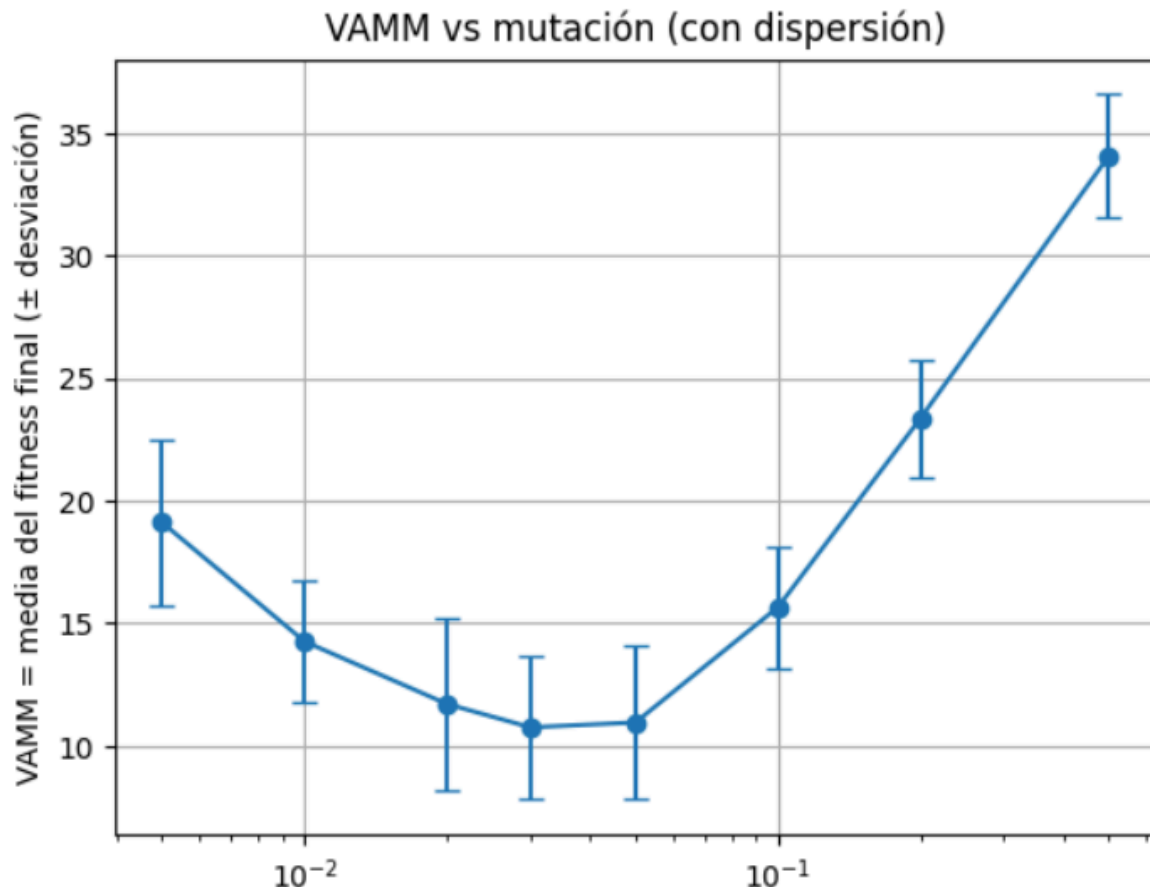
Graficado de la evaluación

Extrayendo los csv de la evaluación para el Sudoku fácil y difícil, aprovecho mis conocimientos como profesor en máster de especialización FP en IA y Big Data donde trabajamos con análisis e ilustración de datos en Pandas entre otros.

Desde Anaconda, accedo a JupyterLab para hacer un notebook y trabajar en las gráficas que puedo hacer. En el código “ga_graph_evaluation.ipynb” expliqué por que debido a que no se llegó en ninguna de las generaciones a un TE diferente de “None”, solo tiene sentido graficar VAMM vs mutación y la curva de progreso.

Resultados experimentales

Influencia de la probabilidad de mutación

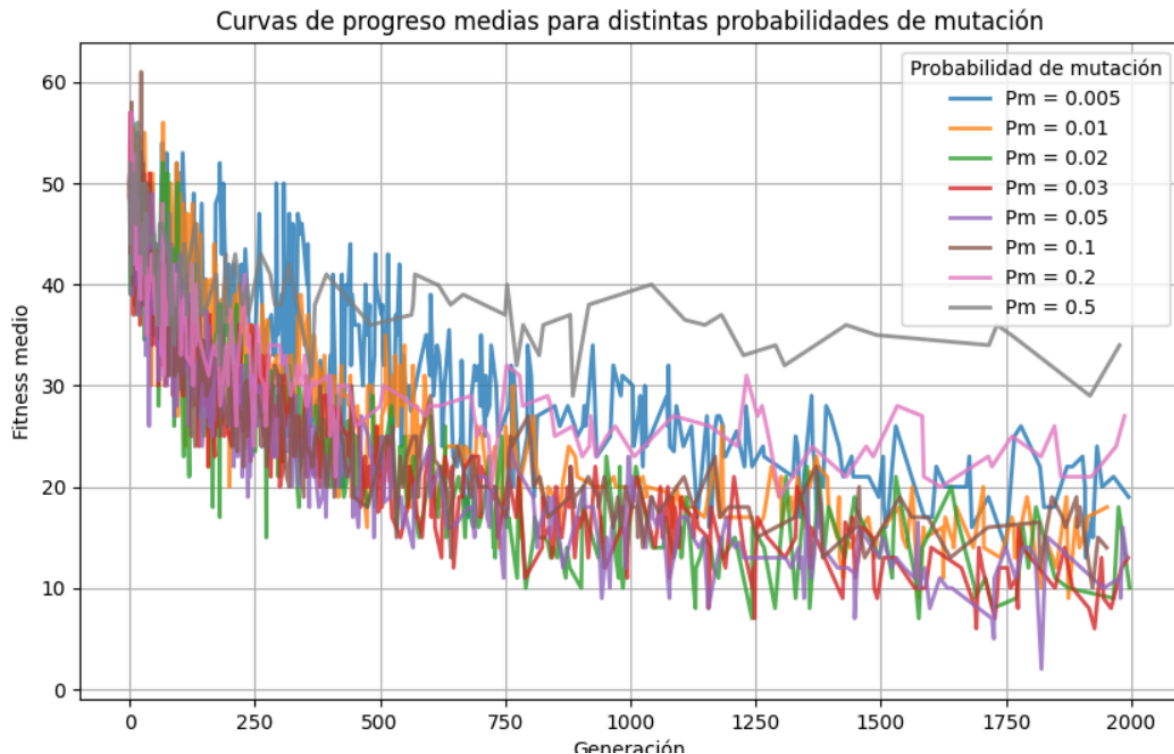


Observamos una mejora progresiva de la VAMM aumentando la mutación a partir de niveles bajos, alcanzando la mejor aptitud media en el rango 0,02 - 0,05.

Sin embargo, cuando se excede este intervalo, el VAMM aumenta drásticamente y la dispersión aumenta, evidenciando una degradación en la estabilidad del sistema para altos valores de mutación.

Los datos confirman que los valores intermedios de la mutación son aquellos que ofrecen mayor eficacia (mejores soluciones) y robustez (consistencia entre ensayos). Esto coincide con lo expuesto en el capítulo 9, que demuestra que el éxito del algoritmo evolutivo depende de mantener un compromiso adecuado entre la capacidad de explorar el espacio de búsqueda y la capacidad de explotar las soluciones encontradas.

Curvas de progreso



En esta gráfica, aunque se vea algo caótica a primera vista, podemos considerar 3 bloques diferentes entre las mutaciones:

Baja mutación (P_m menor o igual a 0.01): Se muestra una mejora rápida en las primeras generaciones pero tiende al estancamiento. Esto es por la falta de exploración que lo deja atrapado en óptimos locales.

Media mutación (P_m entre 0.02 y 0.1): Este espectro da lugar a la arquitectura más robusta. La disminución de fitness es más estable que la de los 2 grupos indicando un equilibrio adecuado entre exploración y explotación.

Alta mutación (P_m a partir de 0.2): La evolución con este hiperparámetro pierde la capacidad de converger lo que hace que el fitness medio se mantenga alto, dejando de ser selectivo el modelo pareciéndose más a un modelo aleatorio.

Discusión

Los resultados confirman que no existe un único valor óptimo de probabilidad de mutación válido para todos los problemas. Para instancias sencillas, una mutación baja favorece una convergencia rápida, mientras que para instancias complejas es necesario aumentar el grado de exploración. Esto es dado a que comparando los valores entre los resultados del Sudoku fácil y difícil observamos que los VAMM se parecen mucho y que es más o menos proporcional la complejidad de un modelo a resolver con el número necesario de población.

Esto está en consonancia con la teoría de los algoritmos evolutivos y con el principio de que la configuración de parámetros debe adaptarse a la estructura y dificultad del problema.

En añadido, me habría gustado experimentar fuera de los límites de los modelos de las etapas del esquema canónico especificados por el enunciado, tratando de buscar el mejor modelo para la resolución del Sudoku base, o que al menos contara el hecho de esta búsqueda.

Conclusiones

En esta actividad se ha implementado y evaluado un algoritmo genético para resolver el problema del Sudoku, siguiendo rigurosamente el esquema canónico descrito en el libro de referencia y los parámetros especificados en el enunciado.

Las principales conclusiones son:

- El algoritmo genético es capaz de resolver instancias de Sudoku.
- La probabilidad de mutación tiene una influencia crítica en el rendimiento.
- Problemas más complejos requieren mayor exploración.
- La evaluación estadística es imprescindible para extraer conclusiones fiables.

En conclusión, esta práctica ha facilitado la aplicación de integrar los conceptos teóricos de computación evolutiva y comprender la importancia del análisis científico experimental en el campo.